GramExpress Project Documentation

Repository Documentation Source

April 4, 2026

Contents

1 Introduction **4**

2 Project Goals **4**

3 Technology Stack **4**

4 Repository Structure **5**
- 4.1 High-Level Responsibilities . . . 5

5 Runtime Architecture **6**
- 5.1 Application Pattern . . . 6
- 5.2 Role-Aware Behavior . . . 6

6 Configuration **6**
- 6.1 Environment Loading . . . 6
- 6.2 Core Settings . . . 6
- 6.3 Environment Variables Used by the Code . . . 7

7 Domain Model **7**
- 7.1 Reference Enums . . . 7
- 7.2 Abstract Models . . . 8
  - 7.2.1 TimeStampedModel . . . 8
  - 7.2.2 LocationMixin . . . 8
- 7.3 Primary Business Models . . . 8
  - 7.3.1 CustomerProfile . . . 8
  - 7.3.2 ShopOwnerProfile . . . 9
  - 7.3.3 RiderProfile . . . 9
  - 7.3.4 Shop . . . 9
  - 7.3.5 Product . . . 10
  - 7.3.6 CheckoutSession . . . 10
  - 7.3.7 Order . . . 10
  - 7.3.8 OrderItem . . . 11
  - 7.3.9 Notification . . . 11
  - 7.3.10 EmailOtpToken . . . 11
  - 7.3.11 AuthOtpToken . . . 11

8 Authentication and Identity Design **12**
8.1 User Resolution . . . 12
8.2 Login Paths . . . 12
8.3 Registration Path . . . 12
8.4 Google Sign-In . . . 12

9 Forms Layer **12**
9.1 Design . . . 12
9.2 Important Forms . . . 13

10 Routing and Functional Areas **13**
10.1 Project URL Configuration . . . 13
10.2 Main Route Groups . . . 13

11 Major User Workflows **14**
11.1 Customer Workflow . . . 14
11.2 Shop Workflow . . . 14
11.3 Rider Workflow . . . 15
11.4 Notification Workflow . . . 15

12 System Flowchart **15**

13 Checkout, Payment, and Delivery Logic **15**
13.1 Cart Handling . . . 15
13.2 Checkout Session . . . 16
13.3 Order Lifecycle . . . 17
13.4 COD and Razorpay Support . . . 17
13.5 Delivery OTP . . . 17

14 OTP Delivery and Messaging **18**
14.1 Email OTP . . . 18
14.2 SMS OTP . . . 18
14.3 Operational Implication . . . 18

15 Templates and Frontend Composition **18**
15.1 Rendering Approach . . . 18
15.2 Template Coverage . . . 18
15.3 Styling . . . 18

16 Progressive Web App Support **19**

17 Seed Data **19**
17.1 Purpose . . . 19
17.2 Objects Created . . . 19
17.3 Demo Credentials . . . 19

18 Database and Migrations **20**
18.1 Database Choice . . . 20
18.2 Migration History . . . 20

19 Testing **20**
19.1 Test Coverage Themes 20
19.2 Testing Command 21

20 Local Setup and Run Instructions **21**
20.1 Initial Setup 21
20.2 Run the Development Server 21
20.3 Operational Note 21

21 Strengths of the Current Implementation **21**

22 Current Limitations **22**

23 Recommended Next Steps **22**

24 Conclusion **22**

1 Introduction

GramExpress is a mobile-first Django application that models a hyperlocal delivery workflow for three primary business roles: customer, store owner, and rider. The repository is implemented as a server-rendered web application, not as a split frontend and backend monorepo. Django handles the application logic, templating, static asset delivery in development, authentication, and database access.

This document is provided in LaTeX source format only. It is intended to describe the complete current project as represented in the repository, including architecture, module layout, domain model, authentication flows, checkout and delivery flows, configuration, seeded data, and test coverage.

2 Project Goals

The system is designed to demonstrate the core capabilities of a local commerce and last-mile delivery platform:

- unified account access using phone or email;
- OTP-assisted registration and login flows;
- distinct dashboards for customer, shop owner, and rider roles;
- product browsing, session cart management, and checkout;
- order tracking, rider handoff OTP, and delivery completion;
- notification-driven user updates;
- an installable progressive web app shell for mobile usage.

3 Technology Stack

Layer	**Implementation**
Backend framework	Django 5.2.x
Language	Python 3.11
Database	SQLite for local development
Frontend	Django templates with custom CSS
Static assets	`static/core/`
Authentication	Django auth plus custom role-profile mapping
OTP delivery	Email backend and SMS backend with console or Twilio option
Payments	COD flow plus Razorpay-related checkout and settlement fields
PWA support	Manifest endpoint and service worker endpoint

4 Repository Structure

The repository is centered around one Django project package and one main app package:

```
gramexpress/
|- manage.py
|- requirements.txt
|- README.md
|- core/
| |- admin.py
| |- apps.py
| |- context_processors.py
| |- forms.py
| |- models.py
| |- tests.py
| |- urls.py
| |- views.py
| |- management/commands/seed_demo.py
| `- migrations/
|- gramexpress/
| |- settings.py
| |- urls.py
| |- asgi.py
| `- wsgi.py
|- templates/core/
`- static/core/
```

4.1 High-Level Responsibilities

- `gramexpress/settings.py` defines the runtime configuration, environment-variable loading, installed apps, middleware, and storage settings.
- `core/models.py` contains the business data model and the domain enums used throughout the system.
- `core/views.py` contains nearly all application behavior, including authentication, cart logic, checkout, order state changes, notifications, and PWA endpoints.
- `core/forms.py` defines styled forms for login, OTP verification, registration, profile updates, products, ratings, and rider location updates.
- `core/urls.py` exposes the route map for customer, shop, rider, payment, support, and notification workflows.
- `templates/core/` contains the server-rendered user interface for all roles.
- `static/core/styles.css` provides the main visual styling.
- `core/management/commands/seed_demo.py` creates demo accounts and sample data for local use.

5 Runtime Architecture

5.1 Application Pattern

GramExpress follows a classic Django server-rendered architecture:

1. the browser requests a page or submits a form;
2. Django routes the request through `gramexpress/urls.py` and `core/urls.py`;
3. view functions in `core/views.py` perform validation, read or mutate model data, and prepare context;
4. Django renders templates in `templates/core/`;
5. static assets are served locally during development;
6. SQLite stores all relational data in `db.sqlite3`.

5.2 Role-Aware Behavior

The application is role-aware at the business-model layer rather than by relying only on the built-in Django groups system. A Django user can be associated with one of several profile models:

- `CustomerProfile`
- `ShopOwnerProfile`
- `RiderProfile`

This approach lets the code attach role-specific data, dashboard behavior, and workflows while still using Django's native session authentication.

6 Configuration

6.1 Environment Loading

The project loads environment variables from a local `.env` file via `python-dotenv`. This file is read from the project base directory during Django startup.

6.2 Core Settings

Important settings in the current codebase include:

- `DEBUG`
- `SECRET_KEY`
- `ALLOWED_HOSTS`
- `CSRF_TRUSTED_ORIGINS`
- SQLite database configuration

- static and media path configuration
- login and redirect URLs
- OTP expiry duration
- email backend configuration
- SMS backend configuration
- Google sign-in configuration
- Google Maps keys
- Razorpay payment and settlement configuration

6.3 Environment Variables Used by the Code

Variable	Purpose
`SECRET_KEY`	Django application secret key
`DEBUG`	Development mode toggle
`GRAMEXPRESS_APP_NAME`	Branding label used by the application
`GOOGLE_MAPS_BROWSER_API_KEY`	Browser-side maps integration
`GOOGLE_MAPS_EMBED_API_KEY`	Embedded maps integration
`GOOGLE_CLIENT_ID`	Google sign-in token audience validation
`RAZORPAY_KEY_ID`	Razorpay payment integration key
`RAZORPAY_KEY_SECRET`	Razorpay secret key
`RAZORPAY_WEBHOOK_SECRET`	Razorpay webhook verification secret
`RAZORPAY_SETTLEMENT_QR_IMAGE_URL`	QR image used for COD settlement flow
`RAZORPAY_SETTLEMENT_UPI_ID`	UPI target for settlement collection
`DEFAULT_FROM_EMAIL`	Sender for email notifications and OTP
`EMAIL_BACKEND`	Email delivery backend
`EMAIL_HOST`, `EMAIL_PORT`, `EMAIL_HOST_USER`, `EMAIL_HOST_PASSWORD`, `EMAIL_USE_TLS`	SMTP configuration
`OTP_EXPIRY_MINUTES`	Expiry window for auth OTPs
`SMS_BACKEND`	SMS delivery mode, including console or Twilio
`SMS_FROM`	Sender number for Twilio SMS
`TWILIO_ACCOUNT_SID`, `TWILIO_AUTH_TOKEN`	Twilio credentials

7 Domain Model

7.1 Reference Enums

The code defines several `TextChoices` enums that shape business behavior:

- `ApprovalStatus`: pending, approved, rejected

- `ShopType`: kirana, medical, bakery
- `VehicleType`: electric, petrol
- `OrderStatus`: pending, confirmed, packed, out_for_delivery, delivered, cancelled
- `PaymentMethod`: COD, Razorpay
- `PaymentStatus`: pending, paid, failed
- `CodCollectionMode`: online, cash
- `SettlementStatus`: not_required, qr_ready, paid, failed
- `RoleType`: customer, shop, rider, admin
- `NotificationType`: order, store, rider, payment, promo, system
- `OtpPurpose`: register, login_email
- `OtpChannel`: email, SMS

7.2 Abstract Models

7.2.1 TimeStampedModel

Provides `created_at` and `updated_at`. Many business objects inherit from this base so the system can group notifications, order timelines, and recent activity by time.

7.2.2 LocationMixin

Provides:

- latitude;
- longitude;
- a computed Google Maps search URL;
- a short coordinate string representation.

7.3 Primary Business Models

7.3.1 CustomerProfile

Stores the customer-specific identity and delivery details:

- linked Django user;
- full name, phone, and email;
- preferred language;
- address lines, district, and pincode;
- latitude and longitude.

7.3.2 ShopOwnerProfile

Represents a store owner:

- linked Django user;
- full name, phone, and email;
- approval status for moderation.

7.3.3 RiderProfile

Represents a delivery rider:

- linked Django user;
- full name, phone, and email;
- age and vehicle type;
- approval status;
- availability flag;
- service radius and maximum service radius;
- rating;
- coordinates;
- optional photo URL or uploaded file.

7.3.4 Shop

Represents a merchant storefront:

- owner foreign key;
- shop name and auto-generated slug;
- shop type;
- service area and address;
- descriptive text and offer label;
- image URL or uploaded image;
- open or closed state;
- approval status;
- rating;
- coordinates.

7.3.5 Product

Represents a sellable item associated with a shop:

- product name and subtitle;
- category and unit;
- price and stock;
- image URL;
- decorative color value;
- merchandising tag.

7.3.6 CheckoutSession

Captures the state of a checkout attempt before or while orders are finalized:

- customer;
- payment method and payment status;
- amount and currency;
- customer notes and delivery address;
- cart snapshot and cart signature;
- receipt and Razorpay identifiers;
- failure reason;
- completion timestamp.

7.3.7 Order

This is the central transactional model. It stores:

- customer, shop, rider, and checkout session references;
- current lifecycle status;
- payment method, payment status, and payment reference;
- COD collection mode and COD payment link metadata;
- customer payment timestamp and cash confirmation timestamp;
- settlement state and settlement-related fields;
- total amount and delivery fee;
- delivery address and customer notes;
- delivery OTP for handoff;

- cancellation metadata;
- customer and store rating fields;
- delivery completion timestamp.

The model also exposes behavior such as recalculating totals, generating a display ID, reporting tracking labels, and checking cancellation or rating eligibility.

7.3.8 OrderItem

Connects an order to a product and stores:

- quantity;
- unit price at the time of order;
- computed line total.

7.3.9 Notification

Stores role-targeted notifications with:

- optional customer target;
- optional shop owner target;
- optional rider target;
- optional related order;
- notification type;
- title and body;
- read or unread state.

7.3.10 EmailOtpToken

This model represents an earlier email OTP workflow and remains in the codebase for compatibility and continuity.

7.3.11 AuthOtpToken

This is the main OTP model used by the current authentication flows. It stores:

- linked user when available;
- role;
- OTP purpose;
- delivery channel;
- email or phone target;

- 6-digit OTP code;
- expiration timestamp;
- used flag;
- metadata JSON for flow-specific state.

8 Authentication and Identity Design

8.1 User Resolution

The application resolves identities by phone number or email address. Instead of treating the Django username as the only entry point, it searches the profile tables to determine whether the supplied identity maps to a customer, shop owner, or rider. This is important because role drives both dashboard routing and authorization behavior.

8.2 Login Paths

The system supports two major login paths:

1. phone or email plus password through the standard login view;
2. passwordless email OTP through the email OTP flow.

When email-based login is used, the system can require OTP completion before granting full access.

8.3 Registration Path

The registration flow is unified into one role-aware form. The selected account type controls which fields are required. Customer registration captures address and language, shop registration captures shop metadata, and rider registration captures vehicle and location details. Registration includes mobile OTP verification before the account is finalized.

8.4 Google Sign-In

The code includes a Google sign-in verification hook. It validates the Google credential token against the configured Google client ID, then allows the application to connect existing accounts. This is not a full standalone OAuth implementation with dedicated social-auth persistence.

9 Forms Layer

9.1 Design

The forms layer is responsible for validation, field styling, and role-aware required fields. A shared `BaseStyledForm` injects consistent input classes and useful HTML attributes such as placeholder text, autocomplete hints, and mobile-friendly input modes.

9.2 Important Forms

Form	Purpose
`LoginForm`	Identity and password login
`LoginOtpVerifyForm`	Email OTP verification after login
`EmailOtpRequestForm`	Passwordless email OTP request
`EmailOtpVerifyForm`	Passwordless email OTP verification
`UnifiedRegistrationForm`	Role-aware registration for customer, shop, and rider
`CustomerProfileForm`	Customer profile updates
`ShopUpdateForm`	Shop metadata updates
`ProductForm`	Product creation and editing
`CustomerOrderMetaForm`	Payment method and checkout notes
`RatingForm`	Customer rating of an order
`StoreRatingForm`	Store rating of a rider delivery
`RiderLocationForm`	Rider coordinate updates

10 Routing and Functional Areas

10.1 Project URL Configuration

The top-level URL configuration is intentionally minimal:

- `/admin/` for Django admin;
- all remaining routes are delegated to `core.urls`.

10.2 Main Route Groups

Route Group	Purpose
`/auth/...`	login, registration, Google auth, and email OTP
`/customer/...`	customer dashboard, cart, checkout, orders, profile, and order actions
`/shop/...`	shop dashboard, products, settings, and order state transitions
`/rider/...`	rider dashboard, deliveries, availability, location, and handoff actions
`/notifications/...`	notification list, read state changes, and linked navigation
`/orders/...`	order detail and tracking views
`/payments/...`	Razorpay webhook handler
`/manifest.json` and `/service-worker.js`	PWA endpoints

11 Major User Workflows

11.1 Customer Workflow

The customer journey is built around discovery, checkout, and tracking:

1. authenticate or register;
2. browse approved and open shops;
3. inspect store details and products;
4. add items to the session cart;
5. review cart and checkout details;
6. choose payment method;
7. create order records;
8. receive order and rider notifications;
9. view order detail and tracking;
10. confirm COD cash when applicable;
11. reorder or rate completed deliveries.

11.2 Shop Workflow

The store owner workflow is centered on catalog management and order processing:

1. authenticate as a store owner;
2. access the store dashboard;
3. manage store settings and basic metadata;
4. create, edit, and remove products;
5. review incoming orders;
6. update order status from pending toward dispatch;
7. monitor rider assignment and delivery progress;
8. rate rider performance after delivery.

11.3 Rider Workflow

The rider workflow combines availability, dispatch acceptance, and proof-of-handoff:

1. authenticate as an approved rider;
2. toggle availability;
3. inspect delivery queue and assigned work;
4. accept an eligible order;
5. update rider location;
6. confirm pickup when near the shop geofence;
7. send or resend delivery OTP reminders;
8. complete delivery after verifying the customer OTP;
9. return to available status after completion.

11.4 Notification Workflow

Notifications are created for all three major roles and are displayed in a dedicated notification center. Users can:

- view grouped notifications;
- mark a single notification as read;
- mark all notifications as read;
- open a notification and navigate to the relevant screen.

12 System Flowchart

The following flowchart summarizes the main end-to-end interaction across authentication, ordering, fulfillment, and completion:

13 Checkout, Payment, and Delivery Logic

13.1 Cart Handling

Cart state is session-based rather than persisted as a dedicated cart model. This keeps the demo lightweight while still enabling:

- adding products;
- updating quantities;
- clearing the cart;
- validating stock before checkout.

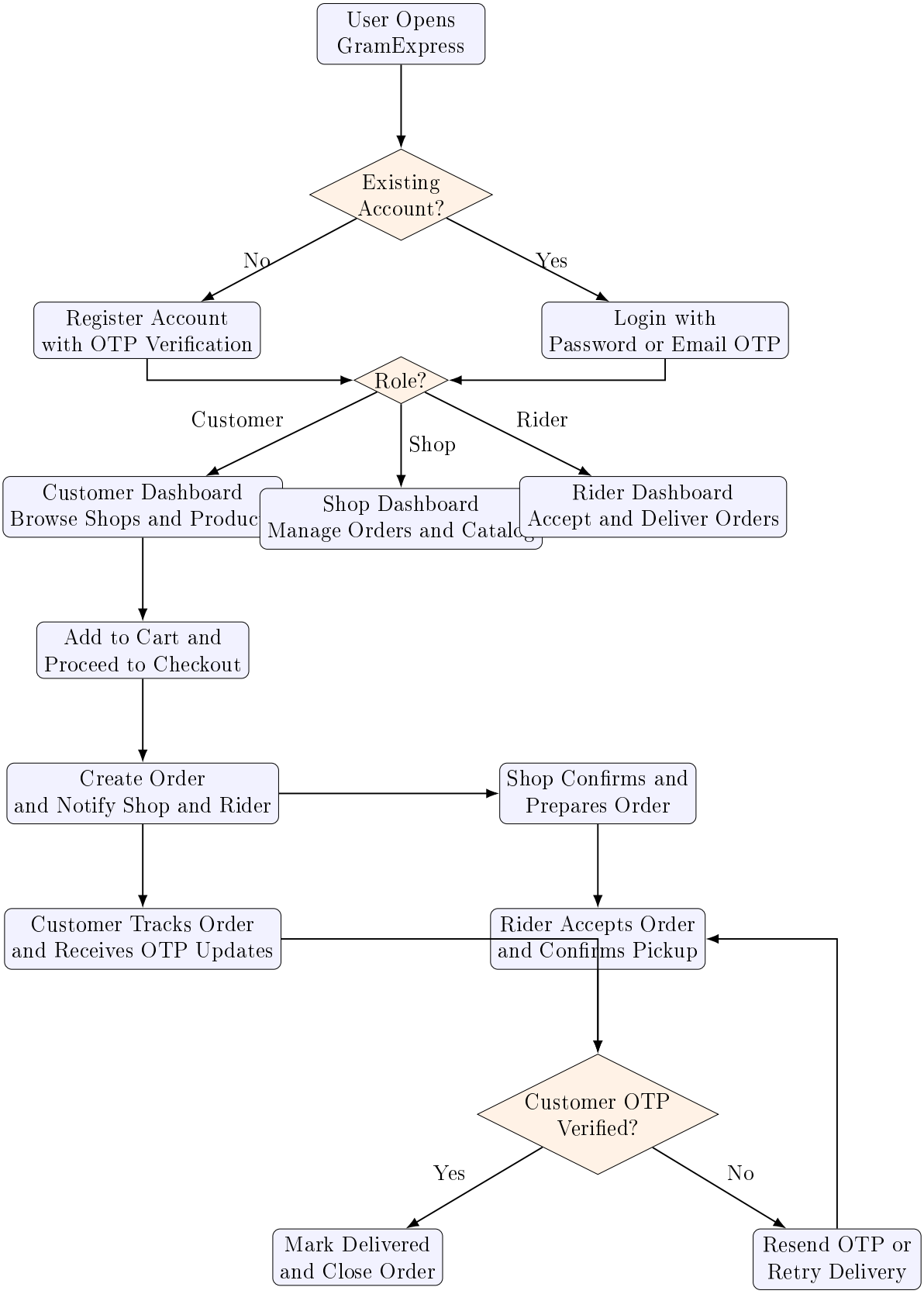

Figure 1: High-level GramExpress workflow from authentication to delivery completion

13.2 Checkout Session

Before or during order creation, the application captures a checkout session snapshot. This structure supports:

- customer checkout review;
- cart signature tracking;
- payment outcome linking;
- retry and failure visibility.

13.3 Order Lifecycle

The main order state machine consists of:

1. `pending`
2. `confirmed`
3. `packed`
4. `out_for_delivery`
5. `delivered`
6. `cancelled`

The customer can cancel only in eligible early states. Riders can move a qualifying order into pickup and then delivery-complete states. The final handoff requires OTP verification.

13.4 COD and Razorpay Support

The codebase includes support for:

- COD as a primary payment method;
- Razorpay-related checkout completion and failure endpoints;
- COD online payment link support;
- settlement metadata for post-delivery reconciliation.

This indicates a hybrid payment design where delivery and settlement can be tracked more deeply than a simple cash-or-online toggle.

13.5 Delivery OTP

Customer handoff OTP is a central trust mechanism in the delivery flow:

- a delivery OTP is stored per order;
- riders can resend a fresh OTP reminder under cooldown rules;
- delivery completion checks OTP validity and equality before accepting the handoff;
- the system also accounts for OTP expiry.

14 OTP Delivery and Messaging

14.1 Email OTP

Email OTP messages are sent using Django's email backend. In local development, the console email backend can be used so the OTP is printed directly in the server terminal instead of being sent to a real mailbox.

14.2 SMS OTP

SMS OTP delivery supports two modes:

- console mode, where the OTP is printed to the server output;
- Twilio mode, where an HTTP request is made to Twilio's message API.

14.3 Operational Implication

With the console backends enabled, local testing becomes simple because both email OTP and mobile OTP can be retrieved directly from the terminal where `runserver` is running.

15 Templates and Frontend Composition

15.1 Rendering Approach

The frontend is not a separate JavaScript application. Instead, the project uses Django templates for all views. This means the backend and frontend are delivered together by the same Django server process.

15.2 Template Coverage

The repository includes dedicated templates for:

- home and login pages;
- registration and email OTP screens;
- customer dashboard, cart, profile, orders, checkout review, and checkout success;
- shop dashboard, products, orders, settings, and start page;
- rider dashboard, deliveries, completed orders, earnings, profile, and start page;
- order detail and order tracking;
- notifications and support;
- a tailwind-theme reference page.

15.3 Styling

Primary styling is maintained in `static/core/styles.css`. The codebase also includes app icons and reference images used by the UI and progressive web app shell.

16 Progressive Web App Support

The application exposes two PWA endpoints:

- `/manifest.json`
- `/service-worker.js`

The service worker caches shell assets and supports installation-oriented mobile usage. Asset versioning is derived from file modification times of selected template and static files, which helps invalidate stale shell caches during development changes.

17 Seed Data

17.1 Purpose

The `seed_demo` management command creates a consistent local environment for demonstrations and testing. This is important because the application has multiple roles and workflows that are difficult to inspect without prepared data.

17.2 Objects Created

The seed script creates or refreshes:

- an admin user;
- a customer user and profile;
- a store owner user and profile;
- a rider user and profile;
- an approved shop;
- sample products;
- a sample order and order items;
- starter notifications.

17.3 Demo Credentials

The repository documentation and seed logic establish the following demo access:

- Admin: `admin / admin123`
- Customer phone: `+919188843299`
- Store phone: `+919900010101`
- Rider phone: `+919900011111`
- Shared non-admin password: `demo12345`

18 Database and Migrations

18.1 Database Choice

The local environment uses SQLite, configured through Django's default database connection. This means there is no separate database server process to launch during development.

18.2 Migration History

The current migration sequence in the repository is:

1. `0001_initial`
2. `0002_customerprofile_user_order_customer_rating_and_more`
3. `0003_authotptoken`
4. `0004_order_cancellation_reason_order_cancelled_by_role`
5. `0005_notification_notification_type`
6. `0006_checkoutsession_order_checkout_session`
7. `0007_order_cash_confirmed_at_order_cod_collection_mode_and_more`

The migration chain shows the project evolving from a simpler transactional core into a richer workflow with OTP, notifications, checkout sessions, COD handling, and settlement support.

19 Testing

19.1 Test Coverage Themes

The repository includes `core/tests.py`, which exercises important behavior such as:

- login and registration rendering;
- role-based redirects;
- cart and checkout operations;
- stock validation;
- notifications;
- order detail and tracking;
- cancellation and reorder behavior;
- registration OTP flow;
- email OTP flow;
- admin approval behavior;
- delivery OTP reminders and expiry handling.

19.2 Testing Command

```
python manage.py test
```

20 Local Setup and Run Instructions

20.1 Initial Setup

```
python -m venv .venv
.venv\Scripts\python -m pip install -r requirements.txt
Copy-Item .env.example .env
.venv\Scripts\python manage.py migrate
.venv\Scripts\python manage.py seed_demo
```

20.2 Run the Development Server

```
.venv\Scripts\python manage.py runserver 0.0.0.0:8000
```

Then open:

```
http://127.0.0.1:8000
```

20.3 Operational Note

Because this project is server-rendered, running the Django development server starts both the backend logic and the frontend UI. SQLite is file-based, so there is no separate database daemon to run.

21 Strengths of the Current Implementation

- end-to-end multi-role demo in a single coherent Django app;
- clear data separation between authentication users and role profiles;
- practical OTP-based login and registration flows;
- customer, store, and rider experiences all exist in the same codebase;
- seeded demo data shortens local onboarding time;
- the PWA shell improves mobile orientation;
- test coverage exists for many business-critical flows.

22 Current Limitations

- SQLite is suitable for local development but not ideal for production scale.
- Many third-party integrations remain optional or partially demo-oriented.
- The application logic is concentrated heavily in `core/views.py`, which increases coupling.
- There is no background job system for async notifications, retries, or cleanup.
- There is no REST API layer for an external mobile client.
- Real-time tracking is not websocket-based.
- File and media handling remain basic.

23 Recommended Next Steps

1. move production deployments to PostgreSQL;
2. separate large view logic into services or modules to improve maintainability;
3. complete the full payment lifecycle around Razorpay and settlement behavior;
4. add stronger inventory reservation and checkout concurrency protections;
5. expand moderation, support, and operational admin tooling;
6. introduce background jobs for OTP delivery, notifications, and payment callbacks;
7. strengthen observability, logging, and audit trails;
8. consider an API layer only if a separate mobile client becomes a product requirement.

24 Conclusion

GramExpress is a substantial Django prototype for hyperlocal commerce. It combines authentication, role-based dashboards, OTP handling, catalog management, session cart checkout, rider-assisted delivery, notifications, and PWA support in one repository. The current implementation is strong as a demonstration platform and a foundation for further productization.

From an engineering perspective, the project already captures many of the hard workflow edges of a local delivery platform: identity resolution, moderation, delivery proof through OTP, stateful orders, and role-aware communication. Its next phase would benefit from modularization, stronger production infrastructure, and completion of external payment and messaging integrations.